Pre-Search Checklist — KG-Backed Coach Dashboard

Complete this before writing code. Save your AI conversation as a reference document.

Project: Future Take-Home · Prepared by: val · Date: 2026-06-04

Phase 1: Define Your Constraints

1. Domain Selection

- **Which domain: healthcare, insurance, finance, legal, or custom?**

  Custom — fitness coaching with clinical adjacency. Coach-facing dashboard generating safe, personalized workouts plus an AI copilot over member context (injuries, biomarkers, labs), so anatomy/injury modeling is healthcare-grade in spirit but data is synthetic only.

- **What specific use cases will you support?**

  Two surfaces per the spec: (A) workout generator from a coach prompt + time window, with interactive graph-driven adjustments (exclude exercises, injury-aware filtering via anatomy hierarchy, equipment substitution) and a provenance trace; (B) copilot with retrieval over the member-context KG: morning brief, adherence/sleep trends, churn risk, charts, chat history.

- **What are the verification requirements for this domain?**

  Safety constraints must be enforced deterministically through graph traversal, not prompt instructions. Every plan ships a provenance trace: why each exercise was chosen, which graph path justified it, and what was filtered for safety. Copilot answers must be grounded in the member's actual data, not invented.

- **What data sources will you need access to?**

  Provided: data/exercises.json (50 exercises with muscle_groups, joints_loaded, movement_patterns, equipment_required) and data/member-context.json (Jordan Rivera: profile, goals, injuries, workout history, adherence, biomarkers, labs, chat history, coach brief). Ontology references (OPE, COPPER, SNOMED CT, PROV-O, SKOS) as curated lookup tables — no live API calls. Any extra data generated synthetically.

2. Scale & Performance

- **Expected query volume?**

  Demo-scale: one reviewer running one coach session over one member. Design for correctness and clarity, not throughput; no load engineering needed.

- **Acceptable latency for responses?**

  Spec: aim for AI responses under ~5s and be reasonable about token efficiency. Decision: stream copilot and generator output token-by-token so perceived latency stays well under that; graph traversal itself is in-process and sub-millisecond at 50 exercises.

- **Concurrent user requirements?**

  One coach (mock auth) reviewing one member. Inferred: no concurrency requirements beyond a single dev server handling parallel SSE streams.

- **Cost constraints for LLM calls?**

  No budget stated; spec asks for reasonable token efficiency. Decision: keep prompts lean by passing resolved graph concepts and pre-filtered candidates to the model rather than the raw catalog; expect pennies per generation on Claude.

3. Reliability Requirements

- **What's the cost of a wrong answer in your domain?**

  An unsafe exercise recommendation (e.g., loading an injured left knee) is the failure that matters — it is exactly what the evaluation targets. Secondary cost: a hallucinated copilot answer about member data undermines the grounding requirement. Both are assessment-failing, not just user-harming.

- **What verification is non-negotiable?**

  The safety decision must be a graph traversal: injured joint expanded through part-of to sub-structures, equipment availability via requires edges, explicit exclusions, contraindicated-for edges. The LLM never gets to override the filter; it only selects and structures from the pre-filtered candidate set.

- **Human-in-the-loop requirements?**

  The coach is the human in the loop by design — every plan is presented to a coach who can interactively adjust it; nothing goes straight to a member.

- **Audit/compliance needs?**

  No regulatory regime (synthetic data, explicit 'never use real member data'). Auditability is still core: PROV-O-style provenance records why each exercise was selected, and the trace renders in the UI per plan.

4. Team & Skill Constraints

- **Familiarity with agent frameworks?**

  Solo build. Decision: hand-rolled Anthropic tool-use loop, no framework — full control, easy to trace, nothing to debug but my own code in a 1-day timebox.

- **Experience with your chosen domain?**

  Inferred: enough fitness-domain fluency to model the anatomy hierarchy and exercise taxonomy from the dataset's own fields (19 muscle groups, 9 joints, 36 movement patterns, 32 equipment types); SNOMED/OPE codes attached via a curated mapping table rather than expert ontology work.

- **Comfort with eval/testing frameworks?**

  pytest for unit tests (resolver + safety filter are mandated); a small custom eval script over a labeled query set for resolver accuracy and safety-filter correctness. No external eval platform.

Phase 2: Architecture Discovery

5. Agent Framework Selection

- **LangChain vs LangGraph vs CrewAI vs custom?**

Custom — a hand-rolled orchestrator over the Anthropic API. The spec rewards an effective multi-agent workflow and explainability; a thin custom loop makes every tool call and graph query traceable, which feeds the provenance and observability requirements directly.

- **Single agent or multi-agent architecture?**

  Multi-agent, kept shallow: an orchestrator that routes to (1) a concept-resolution step, (2) a deterministic graph-traversal safety filter (not an LLM), (3) a plan-composer agent that structures warmup/main/cooldown from filtered candidates, and (4) a copilot retrieval agent for surface B. Multi-agent orchestration is an explicit nice-to-have in the spec.

- **State management requirements?**

  Per-session conversation state for the copilot (chat history with images per the spec) and per-generation provenance records. In-memory with JSON persistence — matches the in-process NetworkX graph decision; no external state store.

- **Tool integration complexity?**

  Low-moderate: ~5 internal tools (resolve_concepts, filter_candidates via traversal, compose_plan, query_member_context, render_chart_data). All in-process; only external dependencies are the Anthropic and Voyage APIs.

6. LLM Selection

- **GPT-5 vs Claude vs open source?**

  Claude via the Anthropic API (decided). Voyage AI for embeddings in the resolver's third pass. One generation provider keeps the runtime simple and the README defensible.

- **Function calling support requirements?**

  Required — the orchestrator is a tool-use loop. Claude's tool calling drives concept resolution handoff, member-context retrieval, and chart-data tools; streaming tool use supports the streaming nice-to-have.

- **Context window needs?**

  Modest. Candidate exercise sets after graph filtering are small (≤50 items), member context is one JSON file summarized into retrieval chunks; well under 50k tokens per call. The KG does the compression that long context would otherwise do.

- **Cost per query acceptable?**

  Inferred: a take-home demo costs cents per interaction on Claude Sonnet-class models; acceptable without further controls. Token efficiency addressed by passing graph-resolved concepts, not raw text dumps.

7. Tool Design

- **What tools does your agent need?**

  resolve_concepts (text → canonical graph nodes), traverse_safety_filter (deterministic, code-only — exposed as a step, not an LLM tool), compose_plan, get_member_context (retrieval over KG 2), get_timeseries (adherence/sleep/biomarkers for charts), get_chat_history. Provenance recording wraps the generation path.

- **External API dependencies?**

Anthropic API (generation) and Voyage AI (embeddings) only. SNOMED/OPE grounding is a curated offline lookup table — deliberately no NCI EVS network dependency so the app runs anywhere with two keys.

- **Mock vs real data for development?**

  Synthetic only, per the spec's hard rule. Provided exercises.json and member-context.json are the dataset; any additional members or edge-case fixtures generated synthetically.

- **Error handling per tool?**

  Resolver degrades gracefully: exact → fuzzy → embedding, each with explicit confidence thresholds; below threshold it reports 'no match' with the attempted passes rather than guessing. API failures surface as user-visible errors in the UI; the safety filter is pure code and cannot fail open — empty candidate sets are reported, never bypassed.

8. Observability Strategy

- **LangSmith vs Braintrust vs other?**

  Neither — structured logging built in-house (decided as the observability nice-to-have): every LLM call, tool call, and graph query logged as structured JSON with timing, token counts, and the traversal paths taken. Keeps the one-command-run promise.

- **What metrics matter most?**

  Resolver accuracy per pass (exact/fuzzy/embedding hit rates and confidences), safety-filter correctness (zero contraindicated exercises in output), end-to-end generation latency vs the ~5s target, and token usage per request.

- **Real-time monitoring needs?**

  None for a take-home. The structured trace per request doubles as the demo of observability; surfacing the trace in the UI alongside provenance is the 'real-time' story.

- **Cost tracking requirements?**

  Token counts logged per call; a per-request cost estimate computed from model pricing in the trace. Sufficient to demonstrate the token-efficiency reasoning the spec asks for.

9. Eval Approach

- **How will you measure correctness?**

  Two mandated critical paths: the concept resolver (does messy input land on the right canonical node, does it degrade gracefully?) and the safety filter (does an injured left knee exclude everything loading the knee and its sub-structures via part-of?). Plus equipment-substitution correctness for the no-barbell scenario.

- **Ground truth data sources?**

  Hand-labeled: a query→expected-concept set for the resolver (including misspellings, synonyms like 'pecs', and no-match cases) and expected include/exclude sets per scenario derived from exercises.json fields (joints_loaded, equipment_required). Member-context answers checked against member-context.json directly.

- **Automated vs human evaluation?**

  Automated pytest for the deterministic paths; the eval pipeline (nice-to-have, in scope) scores resolver accuracy and retrieval relevance over the labeled set. Human evaluation is me

eyeballing copilot groundedness against the JSON — acceptable at this scale.

- **CI integration for eval runs?**

  Inferred: a take-home repo doesn't need CI, but a `make test` and `make eval` target demonstrate the discipline; optionally a GitHub Actions workflow running pytest if time allows.

10. Verification Design

- **What claims must be verified?**

  Every exercise in a plan must be justified by a graph path (targets/requires edges) and must survive the safety traversal; every filtered exercise must record why (which injury, equipment, or exclusion edge). Copilot claims about adherence, sleep, biomarkers, or chats must come from KG 2 retrieval, never model memory.

- **Fact-checking data sources?**

  The graphs themselves are the ground truth: KG 1 (movement/clinical, built from exercises.json + curated ontology mappings) and KG 2 (member context from member-context.json). Provenance trace links each claim back to nodes and edges.

- **Confidence thresholds?**

  Explicit per resolver pass (decided, per spec encouragement): exact match = 1.0; fuzzy accepted above a tuned rapidfuzz threshold (~0.85 to start); embedding fallback accepted above a cosine-similarity floor (~0.75 to start); below floor → no-match path with graceful degradation. Thresholds documented in the README with the tuning rationale.

- **Escalation triggers?**

  No-match concepts surface to the coach as 'couldn't map X — did you mean…' rather than silently dropping constraints. If safety filtering empties the candidate pool, the UI says so explicitly instead of relaxing constraints.

Phase 3: Post-Stack Refinement

11. Failure Mode Analysis

- **What happens when tools fail?**

  Anthropic/Voyage outages surface as explicit UI errors with retry; the embedding pass failing degrades the resolver to exact+fuzzy (logged as degraded mode) rather than blocking. Graph operations are in-process and deterministic — no partial-failure modes.

- **How to handle ambiguous queries?**

  Resolver confidence drives behavior: high-confidence matches proceed; mid-confidence matches are used but flagged in the provenance trace; no-match triggers a clarification back to the coach. Ambiguity is never resolved by letting the LLM guess a safety-relevant constraint.

- **Rate limiting and fallback strategies?**

  Inferred: single-user demo needs no rate limiting; basic exponential backoff on 429s from the Anthropic SDK is sufficient.

- **Graceful degradation approach?**

Three layers: resolver pass degradation (embedding → fuzzy → exact-only), constraint-conflict reporting (empty candidate set → explain which constraint eliminated what), and copilot fallback ('that data isn't in this member's context') instead of hallucination.

12. Security Considerations

- **Prompt injection prevention?**

  Member chat history is rendered to the copilot as quoted data, not instructions; the safety filter being pure graph code means injected text cannot un-filter a contraindicated exercise — the highest-stakes decision is structurally immune. Inferred: that argument goes in the README's safety section.

- **Data leakage risks?**

  Minimal by construction — all data is synthetic and the spec forbids real member data. No PII handling needed; member JSON never leaves the local process except as prompt context to the LLM APIs.

- **API key management?**

  ANTHROPIC_API_KEY and VOYAGE_API_KEY via .env, with .env.example committed and .env gitignored. Keys never reach the frontend; all LLM calls go through FastAPI.

- **Audit logging requirements?**

  The structured trace (Section 8) plus PROV-O-style provenance records per generation serve as the audit log; persisted as JSON so a reviewer can replay why any plan looked the way it did.

13. Testing Strategy

- **Unit tests for tools?**

  pytest on the two mandated paths: resolver (exact/fuzzy/embedding cases, threshold boundaries, no-match degradation) and safety filter (left-knee injury excludes knee-loading exercises through the part-of hierarchy; no-barbell drops barbell-only exercises and substitution finds alternatives; explicit 'exclude deadlifts' removes all variations). Chosen because they are the deterministic safety spine — exactly why the spec mandates them.

- **Integration tests for agent flows?**

  One end-to-end test per scripted scenario: injury case, limited-equipment case, and exclusion case — asserting the final plan contains no filtered exercises and ships a provenance trace. LLM composition checked structurally (valid warmup/main/cooldown schema), not string-matched.

- **Adversarial testing approach?**

  Messy-input resolver cases (typos, slang like 'pecs', nonsense terms) and conflicting constraints (injury + equipment limits that nearly empty the pool). Inferred: a prompt-injection case in member chat data attempting to unlock an excluded exercise, demonstrating the traversal is immune.

- **Regression testing setup?**

  The labeled eval set doubles as the regression suite; `make test && make eval` before pushing. No CI gate required for a take-home, optional GitHub Action if time allows.

14. Open Source Planning

- **What will you release?**

  The deliverable is a runnable GitHub repo with a comprehensive staff-engineer-grade README — shared with Future's reviewers, not a public open-source release.

- **Licensing considerations?**

  Out of scope — private take-home repo; no license decision needed beyond not committing proprietary data (all data is synthetic and provided).

- **Documentation requirements?**

  The README is a first-class deliverable: architecture diagram (Mermaid), stack defense, one-command run instructions, how AI was used to build it, challenges/trade-offs, production evaluation strategy, and 2–3 worked examples including one injury case and one limited-equipment case with provenance traces. KG schema documented: node types, edge types, and their meanings.

- **Community engagement plan?**

  Out of scope — see above.

15. Deployment & Operations

- **Hosting approach?**

  Local-only by design: `make dev` (or ./run.sh) starts FastAPI + Vite; no Docker, no cloud. The one-command run is an explicit evaluation criterion.

- **CI/CD for agent updates?**

  Out of scope — single-day take-home with no deployment target; `make test` is the quality gate.

- **Monitoring and alerting?**

  Out of scope for ops; the README's required 'how you'd evaluate this in production' section covers what monitoring would look like: safety-violation counters, resolver confidence drift, latency/token dashboards, and human escalation review.

- **Rollback strategy?**

  Out of scope — see above; git history is the rollback story.

16. Iteration Planning

- **How will you collect user feedback?**

  In-product, the coach's interactive adjustments are the feedback signal — each 'exclude X' or correction is logged with the provenance trace. For the assessment itself, feedback arrives as the review conversation the README is written to defend.

- **Eval-driven improvement cycle?**

  The eval pipeline (resolver accuracy, safety correctness, retrieval relevance) is the regression baseline; threshold tuning and mapping-table fixes are validated against it before merging. Documented in the README as the production improvement loop.

- **Feature prioritization approach?**

Timebox-driven (decided): required build steps 1–9 first; then nice-to-haves in demo-impact order — graph visualization, streaming, observability traces, eval pipeline. Anything at risk gets cut and documented as a trade-off, which the spec explicitly rewards.

- **Long-term maintenance plan?**

  Out of scope as an ongoing commitment, but the README's production-evaluation section sketches it: versioned ontology mapping tables, periodic resolver eval re-runs as the exercise catalog grows, and safety-filter audits when new injury or equipment types are added.